Efficient ResNet Design and Training for CIFAR-10 Under a 5M Parameter Budget

Yonghyeon Shin
New York University, Tandon School of Engineering
CS-GY 6923, Fall 2026 — Kaggle team: Yonghyeon Shin
https://github.com/yonghyeon96/nyu-dl-project

Abstract

This project investigates residual network architecture and training choices for CIFAR-10 classification under a budget of five million trainable parameters. Starting from a half-width, four-stage ResNet-18, I evaluated stronger augmentation, network width and stage organization, training duration, sample mixing, and knowledge distillation. A three-stage network with 4,327,754 parameters, trained from scratch with TrivialAugmentWide and CutMix, achieved 87.9% private leaderboard accuracy and ranked 17th among 81 teams. Its public score was 89.6%. A separate distilled model reached the highest held-out validation accuracy, 97.48%, but did not yield the best competition result. The experiments show that architecture and regularization choices mattered more than exhausting the parameter budget, while also revealing a substantial gap between validation and competition performance. Most configurations were evaluated once; small differences and comparisons involving multiple changes are therefore interpreted cautiously.

1 Introduction

The Efficient ResNet Architecture Challenge required a convolutional classifier with residual connections, no more than 5,000,000 trainable parameters, and training from scratch on the permitted CIFAR-10 data. CIFAR-10 consists of small RGB images from ten object categories [1]. This competition supplied 50,000 labeled training images and 2,000 evaluation images with hidden labels. The evaluation images were divided into fixed public and private sets of 1,000 images each. The private split determined the final ranking.

Residual networks [2] provide a useful starting point because skip connections support optimization while allowing depth, width, and downsampling choices to be adjusted. Under a fixed parameter budget, however, adding capacity is not necessarily the most effective improvement. The experiments addressed three questions: (1) whether the baseline benefited more from capacity or regularization; (2) how stage organization affected accuracy within the budget; and (3) whether improvements on an internal validation split transferred to the competition evaluation set.

The final submission used a single three-stage ResNet with widths 64/128/256 and three residual blocks per stage. It combined strong augmentation, CutMix, 300 training epochs, and test-time augmentation (TTA). The final private score was 0.879, with rank 17/81. The most informative result was that the strongest validation model was not the strongest competition model. This report therefore presents both successful changes and the limitations of using validation accuracy to predict final performance.

Table 1: Final architecture. Output shapes are channels × height × width. Parameter counts include BN and projection shortcuts.

Component	Output	Parameters
Stem: conv, BN, ReLU	$64 \times 32 \times 32$	1,856
Stage 1: 3 blocks	$64 \times 32 \times 32$	221,952
Stage 2: 3 blocks	$128 \times 16 \times 16$	820,992
Stage 3: 3 blocks	$256 \times 8 \times 8$	3,280,384
Pooling + classifier	10	2,570
Total		**4,327,754**

2 Model Architecture

2.1 Baseline and residual block

The baseline was a half-width ResNet-18 with four stages, widths 32/64/128/256, and two BasicBlocks per stage. It contained 2,797,610 trainable parameters. A BasicBlock computes

$$y = \mathrm{ReLU}\big(F(x) + S(x)\big), \tag{1}$$

where F is a sequence of 3×3 convolution, batch normalization (BN), ReLU, 3×3 convolution, and BN. The shortcut S is an identity mapping when shapes match, or a 1×1 convolution followed by BN when the channel count or spatial resolution changes.

2.2 Final three-stage network

The final architecture uses a 3×3 stem with 64 channels and no max-pooling. Each of its three stages contains three BasicBlocks. The first blocks of stages 2 and 3 downsample with stride 2. Global average pooling and a linear ten-class classifier form the head. Convolution weights use Kaiming normal initialization with `fan_out`. Table 1 accounts for all trainable components, including projections, BN, and the classifier.

The main path has 19 convolutional layers and one linear layer. The network uses approximately 86.6% of the parameter budget. Relative to the baseline, it removes the final

4×4-resolution stage and widens earlier stages. Stage count, width, and block count change together, so this is an architectural comparison rather than an isolated test of any one component. Section 4.2 evaluates this design against wider four-stage alternatives.

3 Experimental Setup

3.1 Data and evaluation protocol

Development experiments used a fixed stratified split of 45,000 training and 5,000 validation images, with 500 validation images per class and split seed 2026. Final models were retrained on all 50,000 training images. Consequently, the full-data models have no held-out validation score of their own. Scores from development models are kept separate from full-data leaderboard results throughout this report.

Inputs were normalized using channel means $(0.4914, 0.4822, 0.4465)$ and standard deviations $(0.2470, 0.2435, 0.2616)$, recorded from the 50,000 training images. These statistics therefore include the held-out development images, although their labels were not used for gradient updates in development runs. The training and prediction code uses the competition evaluation images only to produce predictions. No pretrained weights or external training datasets were used.

For each development run, I recorded the highest validation accuracy and the mean validation accuracy over the last ten epochs. The latter reduces dependence on a single favorable epoch, but remains a model-selection metric on the same validation split, not an independent test estimate. Unless explicitly stated otherwise, validation results in the architectural and training comparisons use no TTA.

3.2 Training configuration

Basic augmentation consisted of random cropping with four-pixel padding and horizontal flipping with probability 0.5. Strong augmentation used TrivialAugmentWide [3], selecting one of 14 operations and a random strength per image, with operation ranges following torchvision. CutMix [4] was applied to 50% of batches, with a mixing weight drawn from $\mathrm{Beta}(1,1)$, equivalent to a uniform distribution on $[0,1]$.

The final model used SGD with momentum 0.9, Nesterov acceleration, weight decay 5×10^{-4}, and batch size 256. The learning rate began at 0.1 and followed a per-step cosine decay to zero over 300 epochs, without warmup or restarts. Cosine scheduling follows the scheduling approach associated with SGDR [5], but no warm restarts were used here. The final submission used training seed 2027; a second full-data model used seed 2026.

Training ran on an NVIDIA A100 40GB at NYU HPC using Python 3.9 and PyTorch 2.6.0. The implementation kept the dataset on the GPU as uint8 and performed batched augmentation on the GPU with per-image random parameters. It used bf16 mixed precision and channels-last memory layout. The recorded 300-epoch full-data CutMix runs took 40.2 and 42.0 minutes. Multiple runs shared the GPU, so elapsed times across rounds are not controlled efficiency comparisons.

3.3 Run-to-run variability

Four identical baseline runs obtained best validation accuracies of 94.18%, 93.78%, 93.88%, and 94.34% (mean 94.05%, sample standard deviation about 0.26 percentage points). The standard deviation of the last-ten-epoch means was approximately 0.285 percentage points. Under an independent-run approximation, a difference between two runs would thus have a standard deviation of about $\sqrt{2}(0.285)\approx0.40$ percentage points.

This is a rough scale of baseline variability, not a significance threshold or a noise estimate established for every later configuration. Most settings were trained once. Small improvements are treated as tentative, and combined interventions are identified explicitly.

4 Experiments and Results

The experiments were organized into four rounds. Round 1 changed the baseline. Round 2 changed the TrivialAugmentWide configuration with 200 epochs. Round 3 changed the three-stage model. Round 4 tested combinations, a second seed, and distillation.

4.1 Regularization before added capacity

The baseline reached approximately 99.97% training accuracy but only 94.05% mean best validation accuracy across its four repetitions. This pattern was consistent with overfitting. Simply increasing width to 40/80/160/320 produced 94.10% best validation accuracy, while extending training from 100 to 200 epochs produced 94.34%. Neither provided a clear gain relative to baseline variability.

At 100 epochs, TrivialAugmentWide improved best validation accuracy to 94.72%, and Cutout [6] with size 16 reached 94.52%. Color jitter, exponential moving average (EMA), and label smoothing did not improve the recorded baseline results. Combining TrivialAugmentWide with 200 epochs reached 95.58%, with a last-ten-epoch mean of 95.48%. Table 2 summarizes the first round.

Figure 1 illustrates the learning behavior. Strong augmentation lowered training accuracy on augmented inputs while increasing validation accuracy. Lower training accuracy alone does not prove reduced memorization, because the training task becomes harder; the accompanying validation improvement supports the value of regularization. These results motivated retaining TrivialAugmentWide for subsequent experiments and revisiting capacity and training duration under that setting.

4.2 Architecture under the budget

With TrivialAugmentWide and 200 epochs, the three-stage model reached 96.52% best validation accuracy, compared

Table 2: Baseline variants. All use 100 epochs unless stated. Accuracies are percentages; "Last 10" is the mean over the final ten epochs. Both baseline scores are averaged over four runs.

Setting	Best	Last 10
Baseline (four-run mean)	94.05	93.87
Wider: 40/80/160/320	94.10	93.92
200 epochs	94.34	94.28
Color jitter 0.3	94.06	93.90
EMA 0.999	93.88	93.80
Label smoothing 0.1	93.52	93.35
Cutout 16	94.52	94.36
TrivialAugmentWide	94.72	94.60
TrivialAugmentWide, 200 epochs	**95.58**	**95.48**

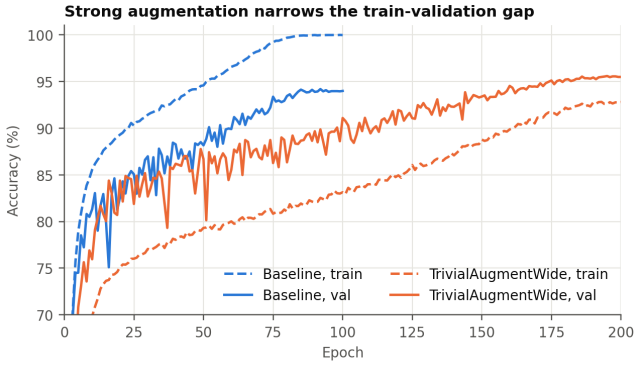


Figure 1: Baseline and strong-augmentation learning curves. Training accuracy is measured on augmented inputs, so its level is not directly comparable across augmentation policies.

with 95.58% for the original four-stage network: a gain of 0.94 percentage points. Its last-ten-epoch mean improved from 95.48% to 96.44%. A wider four-stage network with widths 42/84/168/336 used 4,815,940 parameters but reached 96.02% best validation accuracy and 95.90% over the last ten epochs.

The three-stage network therefore outperformed a larger four-stage alternative in the recorded comparison (Fig. 2). This supports the choice of stage organization and early-stage capacity, but does not isolate which architectural change caused the gain. Extending the four-stage TrivialAugmentWide model from 200 to 300 epochs yielded a small observed improvement, reaching 96.04% best validation accuracy and 95.87% mean accuracy over the last ten epochs. These gains are comparable in scale to the baseline run-to-run variability estimate, but repeated runs of these configurations would be needed to establish a consistent benefit.

Increasing the three-stage widths further to 68/136/272 used 4,884,926 parameters and produced 96.64% best accuracy versus 96.52% for widths 64/128/256. Last-ten-epoch means differed by only 0.05 percentage points (96.49% versus 96.44%). This small single-run difference did not provide convincing evidence that using nearly the entire budget was beneficial. The smaller three-stage design was retained.

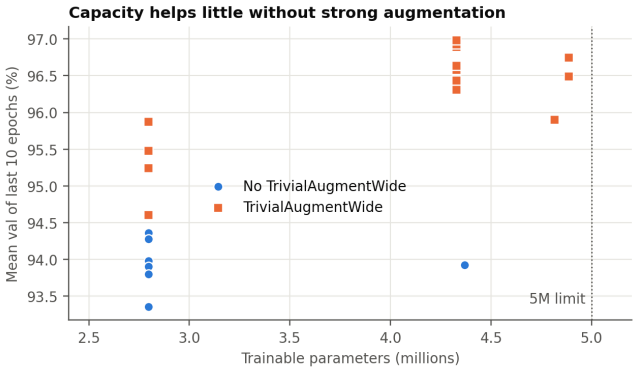


Figure 2: Parameter count versus last-ten-epoch validation accuracy. Architecture and augmentation choices affect the observed relationship; this is not a controlled scaling law.

Table 3: Changes to the three-stage model with TrivialAugmentWide. The reference uses widths 64/128/256 and 200 epochs. Each row changes the listed setting(s).

Setting	Best	Last 10
Reference	96.52	96.44
300 epochs	97.00	**96.93**
CutMix (50% of batches)	97.00	96.89
Wider + 300 epochs	96.80	96.74
SAM ($\rho = 0.05$)	96.84	96.64
Color jitter 0.3	96.66	96.58
Wider: 68/136/272	96.64	96.49
Mixup (50% of batches)	96.38	96.31

4.3 Training duration and sample mixing

Using the three-stage 200-epoch model as a reference, extending training to 300 epochs reached 97.00% best validation accuracy and 96.93% over the last ten epochs. Adding CutMix while retaining 200 epochs also reached 97.00%, with a last-ten-epoch mean of 96.89%. These were the most promising changes in this round (Table 3).

Mixup [7] on half the batches did not improve the reference in this run. Sharpness-aware minimization (SAM) [8] with $\rho = 0.05$ obtained 96.84% best accuracy, at approximately twice the forward/backward work per update. Its gain was modest relative to the baseline variability estimate. Increasing width and duration together reached 96.80%, but changed two factors and did not beat the narrower 300-epoch configuration.

Combining CutMix and 300 epochs yielded 97.10% best validation accuracy for seed 2026 and 96.86% for seed 2027. Their last-ten-epoch means were 96.98% and 96.79%, respectively, a difference of 0.19 percentage points. The combined setting provided little additional validation improvement over the separate interventions, and the seed comparison demonstrated that small differences could change the apparent ordering.

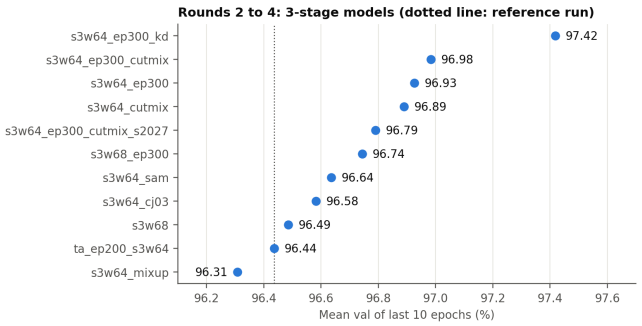


Figure 3: Three-stage configurations compared using last-ten-epoch validation accuracy. The reference is indicated by the dotted line; most settings have one run.

Table 4: Development validation accuracy (%) with TTA. "Shift 1/2" denotes flip plus the corresponding shift setting.

Model	None	Flip	Shift 1	Shift 2
CutMix, seed 2026	97.10	97.30	97.48	97.40
CutMix, seed 2027	96.86	97.14	97.06	97.08
Distillation	97.48	97.32	97.36	97.34

4.4 Knowledge distillation

I also trained a student with the same 4,327,754-parameter architecture using knowledge distillation [9]. Its two teachers were the three-stage 300-epoch model and the three-stage 200-epoch CutMix model. Both teachers were trained from scratch on the allowed data. The student used 300 epochs, temperature 4, and a configured distillation coefficient of 0.9, without CutMix.

Distillation achieved the highest development score: 97.48% best validation accuracy and 97.42% over the last ten epochs. It preserved the student parameter count; the teachers were training-time components. However, the full-data distilled student did not produce the best competition score, as discussed in Section 5.3. This distinction is essential: the validation result establishes a promising development outcome, not superiority on the hidden evaluation data.

5 Inference and Final Evaluation

5.1 Test-time augmentation

TTA averaged softmax probabilities over transformed views. Horizontal-flip TTA used the original image and its flip. The final submission used 18 views: these two orientations at nine spatial shifts, with each axis shifted by -1, 0, or $+1$ pixel and zero padding applied after normalization, corresponding to the per-channel mean color in the original image space. This reuses a single model's weights, increasing inference work without adding trainable parameters.

Flip TTA improved the two CutMix development models by 0.20 and 0.28 percentage points. Additional shifts were less consistent (Table 4). The final shift-one policy was chosen from the seed-2026 development model, where it reached

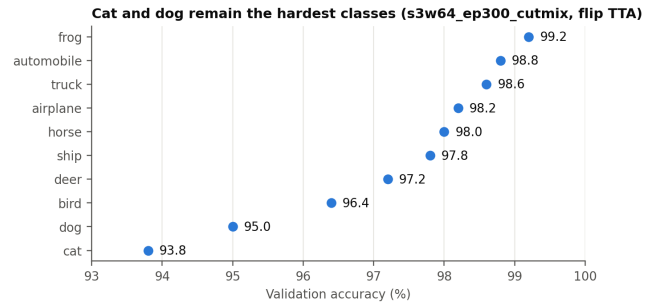


Figure 4: Class-wise validation accuracy for the CutMix 300-epoch development model (seed 2026) with flip TTA. Each class has 500 held-out training images.

97.48%. It did not improve the other two models relative to their strongest listed inference setting. The result therefore does not support a general claim that more test-time views improve accuracy.

A separate validation-only ensemble check averaged predictions from three models, all using flip TTA: the three-stage 300-epoch model (97.02%), the three-stage 200-epoch CutMix model (97.36%), and the CutMix 300-epoch model with seed 2026 (97.30%). Averaging their predictions reached 97.56%, or 122 errors, compared with 97.36%, or 132 errors, for the best single model, the three-stage 200-epoch CutMix model. This multi-model ensemble was not submitted because of the competition's 5M model-budget constraint. The reported final result uses one model.

5.2 Class-wise validation errors

For the CutMix 300-epoch development model (seed 2026) with flip TTA, cat and dog were the least accurate classes at 93.8% and 95.0%. Frog and automobile reached 99.2% and 98.8%. Cat/dog confusions accounted for 36 of the model's 135 validation errors. These observations identify where this particular model struggled; they do not establish the class-wise error distribution on the hidden evaluation set.

5.3 Competition results

Table 5 reports all eight recorded submissions. Submissions 1–5 used development models trained on 45,000 images, with best-validation checkpoints and flip TTA. Submissions 6–8 used final checkpoints from full-data training. Two submissions were selected for final ranking: the full-data CutMix models with seeds 2026 and 2027. Seed 2027 achieved the final private score of 87.9% and rank 17/81, despite its public score being slightly lower than seed 2026's.

The private score improved from 80.5% for the baseline submission to 87.9% for the final model. On all 2,000 evaluation images, accuracy increased from 82.10% to 88.75%. This aggregate score is useful for retrospective analysis, but the official final ranking uses private accuracy alone. Figure 5 shows the development trajectory alongside the leaderboard results.

Two comparisons illustrate why validation and public scores

Table 5: Recorded Kaggle submissions. All scores are percentages. Validation scores are without TTA and belong only to the 45k development runs. "All" is the equal-weight mean of public and private scores, computed from the source CSV without rounding to three decimals. *Selected for final ranking.

ID	Model / training setting	Train size	TTA	Val.	Public	Private	All
1	Baseline, 100 epochs	45,000	Flip	94.18	83.7	80.5	82.10
2	TrivialAugmentWide, 200 epochs	45,000	Flip	95.58	86.4	83.6	85.00
3	3 stages, 200 epochs	45,000	Flip	96.52	87.2	84.5	85.85
4	3 stages, 300 epochs	45,000	Flip	97.00	87.3	84.1	85.70
5	3 stages, CutMix, 200 epochs	45,000	Flip	97.00	87.3	87.1	87.20
6*	CutMix, 300 epochs, seed 2026	50,000	Flip + shift 1	—	89.8	87.1	88.45
7*	**CutMix, 300 epochs, seed 2027**	**50,000**	Flip + shift 1	—	**89.6**	**87.9**	**88.75**
8	Distillation, 300 epochs	50,000	None	—	87.5	85.2	86.35

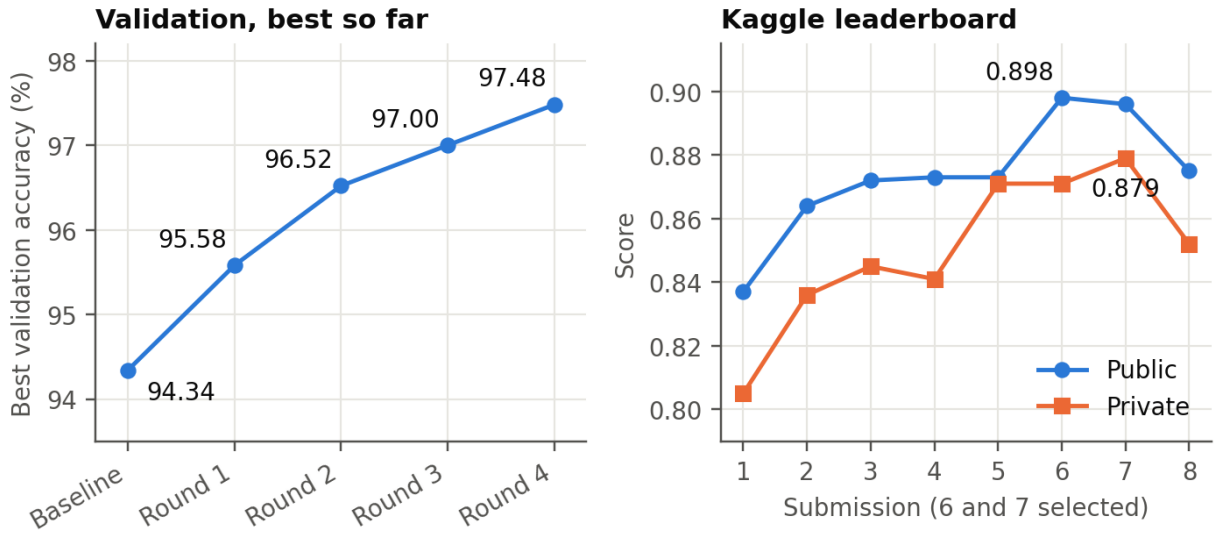


Figure 5: Development progress and competition scores. Left: the highest single-run validation accuracy up to each round. The baseline point (94.34%) is the best of the four repeated baseline runs, not the submitted run (94.18%) or the four-run mean (94.05%). Right: public and private leaderboard scores by submission. Submissions 6 and 7 were selected for final ranking. Private results are analyzed retrospectively.

alone were insufficient. Submissions 4 and 5 both had 97.00% validation accuracy and 87.3% public accuracy, yet private accuracy was 84.1% versus 87.1%. The CutMix run also had fewer epochs, so this is not a perfectly isolated CutMix ablation. Nevertheless, the competition result favored that training configuration even though the two commonly observed scores tied.

Similarly, distillation produced the strongest development validation result, but its full-data submission achieved only 85.2% private accuracy and 86.35% across all evaluation images. It was submitted without CutMix or TTA, whereas the final CutMix models used both. The difference cannot be attributed to distillation alone. It demonstrates that the best development configuration did not automatically identify the best submitted pipeline.

6 Discussion and Limitations

6.1 What the experiments support

The first practical finding is that stronger augmentation unlocked gains that additional capacity and training alone had not provided. The original baseline nearly fit its training data perfectly. Once TrivialAugmentWide was introduced, longer training and the three-stage architecture became more productive. The evidence favors addressing regularization together with architecture, rather than treating parameter count as a direct predictor of performance.

Second, distributing capacity across stages mattered within the tested configurations. The selected three-stage model outperformed a larger four-stage alternative, while widening it further offered only a small, uncertain benefit. The final network left part of the allowed budget unused without sacrificing a demonstrated improvement.

Third, model selection required caution near the top of the validation range. The internal validation scores of the five development models submitted to Kaggle exceeded their

aggregate competition accuracies by roughly 9.8–12.1 percentage points. This gap is consistent with a harder evaluation set or a distribution mismatch, but neither cause was established. Dataset construction, sampling, and other possible explanations were not resolved by these experiments. Private results became available after the competition and are used here for retrospective interpretation, not presented as information available during development.

6.2 Limits of attribution and measurement

Most settings were trained only once, and GPU execution was not fully deterministic. The repeated baseline provides a useful variability reference, but not confidence intervals for the final architectures. Repeated selection on a fixed validation split can also make its highest scores optimistic. Last-ten-epoch averages reduce sensitivity to the chosen epoch but do not eliminate this selection effect.

Several comparisons changed multiple factors. The three-stage architecture altered widths, block counts, and downsampling structure together. Relative to submission 5, full-data submissions 6 and 7 changed training set size, duration, TTA, and checkpoint selection (best-validation checkpoint versus final checkpoint), improving aggregate evaluation accuracy from 87.20% to 88.45% and 88.75%. These gains cannot be assigned separately to the additional 5,000 training images, 100 epochs, shift TTA, or checkpoint selection. The distilled submission likewise differs from the final pipeline in more than its training objective.

Each leaderboard half contains only 1,000 examples, so one changed prediction moves its accuracy by 0.1 percentage points. Small differences between submissions should not be treated as robust rankings across repeated training or new test samples. The final score and rank describe the realized competition outcome. More repetitions, matched inference settings, and additional training-only validation splits would be needed to establish more general conclusions.

7 Reproducibility

The public repository linked on the first page contains model definitions, training and prediction code, preprocessing and augmentation, configurations, per-run results, and an experiment log. The final configuration records the architecture, optimizer, schedule, batch size, seed, and 300-epoch full-data training setup. The following commands reproduce the intended workflow:

```
python -m src.model --config configs/final.json
python train.py --config configs/final.json
python predict.py --ckpt \
  runs/final_<time>/final.pt --tta --shift 1
```

The first command prints 4,327,754 trainable parameters. In the prediction command, `<time>` is replaced with the directory identifier produced by the training run. The model is trained from scratch; prediction uses its saved final checkpoint. Run logs and configurations document the reported experiments, while slight numerical variation remains possible when training is repeated.

8 Conclusion

A three-stage ResNet with 4,327,754 trainable parameters achieved 87.9% private leaderboard accuracy and ranked 17th of 81 teams. Strong augmentation, a revised stage structure, and CutMix formed the most effective submitted pipeline. Filling the remaining parameter budget did not yield a clear validation benefit. Knowledge distillation improved validation accuracy to 97.48%, but its full-data submission did not outperform the final CutMix model. The project highlights both the value of structured experimentation under a parameter constraint and the need to distinguish development improvements from generalization on a hidden evaluation set.

References

[1] Alex Krizhevsky. Learning Multiple Layers of Features from Tiny Images. *Technical Report, University of Toronto*, 2009.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[3] Samuel G. Müller and Frank Hutter. TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[4] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[5] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic Gradient Descent with Warm Restarts. In *International Conference on Learning Representations (ICLR)*, 2017.

[6] Terrance DeVries and Graham W. Taylor. Improved Regularization of Convolutional Neural Networks with Cutout. *arXiv preprint arXiv:1708.04552*, 2017.

[7] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond Empirical Risk Minimization. In *International Conference on Learning Representations (ICLR)*, 2018.

[8] Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. Sharpness-Aware Minimization for Efficiently Improving Generalization. In *International Conference on Learning Representations (ICLR)*, 2021.

[9] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*, 2015.